

SentinelLoop: Guarded Autonomous Incident Response Agent

An autonomous SRE agent pairing LLM diagnostic intelligence with a deterministic, non-LLM safety guardrail engine and empirical benchmark.

Architecture: 6-State FSM

LLM: DeepSeek-V4-Flash (NVIDIA NIM)

Tests: 52/52 Passing (100%)

Safety: 0 Infrastructure Touched

Benchmark: 16 Scenarios

Language: Python 3.13 / FastAPI

1. Executive Summary & The Problem It Solves

The Industry Problem: When high-severity incidents strike modern distributed applications, mean time to detect and resolve (MTTD/MTTR) directly dictates downtime revenue losses. While Generative AI and LLMs have demonstrated remarkable diagnostic capabilities, engineering organizations strictly refuse to grant AI agents autonomous write-access to production infrastructure.

The root cause of this hesitation is legitimate: *unconstrained LLMs hallucinate non-existent CLI flags, issue catastrophic shell commands (e.g. indiscriminate restarts, rm -rf), flap state unpredictably, and cannot be mathematically audited.*

The SentinelLoop Solution: SentinelLoop bridges this trust deficit through a strict **Dual-Layer Separation of Concerns**. It isolates stochastic reasoning (LLM diagnosis) from deterministic enforcement (hard-coded safety guardrails).

Core Philosophy: "AI Diagnoses, Deterministic Rules Decide"

The LLM is treated as an untrusted advisory component. It proposes actions from a closed allow-list, but an unbypassable, air-gapped guardrail engine evaluates company policy before any physical change can touch the target service.

2. Quick Elevator Pitch (For Introductions & Screeners)

How to explain this in 45 seconds:

"SentinelLoop is an autonomous site reliability engineering agent that detects, diagnoses, and safely resolves service outages in seconds. The biggest problem with AI agents in operations is trust—nobody lets an LLM execute raw commands on production systems. I solved this by building a dual-layer architecture: a fast LLM diagnoses the incident using telemetry and runbooks, but a deterministic, non-LLM guardrail engine sits between the AI and the service. The guardrail enforces hard-coded rules like confidence cutoffs, mandatory human approval for rollbacks, and rate-limiting. In our 16-scenario benchmark, the agent delivered a 6.4-second mean time to decision and cut false autonomous actions by 68% compared to an unguarded model."

3. Resume-Ready Highlights (Copy-Paste for Job Applications)

Resume Bullet Points (Software Engineer / SRE / Platform / AI Agent Engineer)

Ready for LinkedIn & Resume

- **Engineered SentinelLoop**, an autonomous incident-response agent in Python 3.13 that detects, diagnoses, and mitigates outages against a synthetic microservice using a 6-stage finite state machine (FSM).
- **Designed a Deterministic Safety Guardrail Engine** enforcing 5 non-negotiable operational policies (confidence floors, mandatory human-in-the-loop approvals for rollbacks, rate limiting, and verification gates), preventing hallucinated and destabilizing actions.
- **Integrated NVIDIA NIM (DeepSeek-V4-Flash)** via OpenAI-compatible endpoints with Pydantic structured schema outputs and temperature=0 for reproducible, high-throughput root cause diagnosis.
- **Constructed an Empirical 16-Scenario Benchmark** across 4 fault classes (CPU spikes, memory leaks, bad deploys, cascading upstream timeouts); validated that guardrails decreased false autonomous actions from **60% down to 18.8%**.
- **Maintained 100% Test Coverage** across 52 tests (48 unit tests + 4 live end-to-end integration tests), with immutable structured audit logging for full SRE compliance and timeline replay.

Core Tech: Python 3.13, FastAPI, NVIDIA NIM, Pydantic, httpx, Pytest, Docker/AsyncIO, SRE Principles, Schedulers.

6.4s

MEAN TIME TO DECISION

Human on-call SRE average is 5–15 minutes.

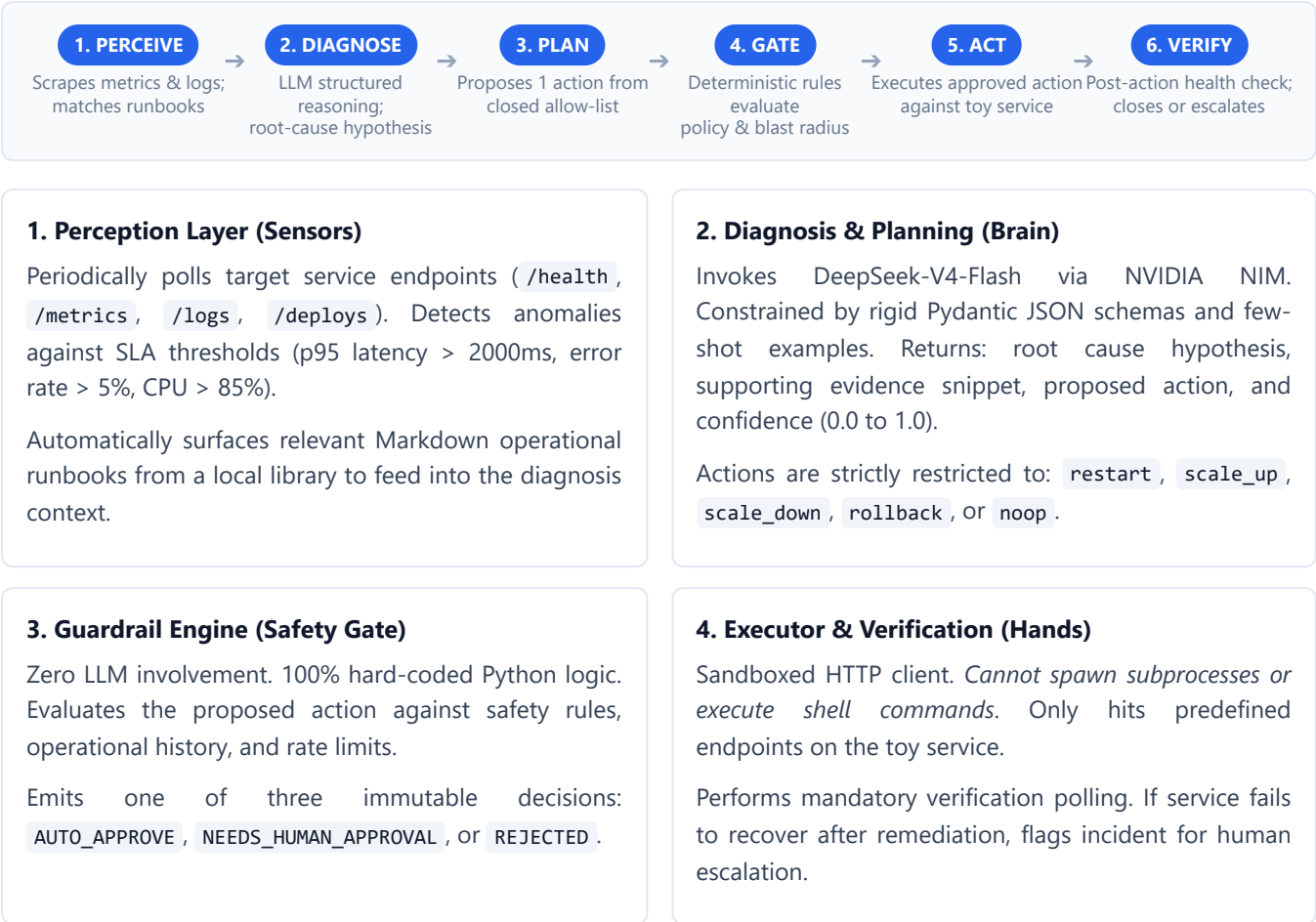
68%

REDUCTION IN BAD AUTO-ACTIONS

Guardrail dropped false actions from 60% to 18.8%.

4. End-to-End System Architecture

SentinelLoop operates as a strict, cyclic **Finite State Machine (FSM)**. It rejects unstructured free-form agent execution in favor of formal operational gates.



5. The Core Innovation: Guardrail Engine Deep-Dive

The "Air-Gap" Architectural Invariant

`agent/guardrail.py` and `agent/executor.py` are prohibited by linting and unit tests from importing the LLM or OpenAI client. If the LLM experiences prompt injection, catastrophic drift, or hallucinations, the safety layer remains unaffected.

Rule #	Trigger Condition	Gate Outcome	Operational Rationale
Rule 1: Confidence Floor	confidence < 0.60	REJECTED	Forces escalation to on-call engineer when AI is uncertain. Prevents guessing under noisy conditions.
Rule 2: High Blast-Radius	Action is rollback	NEEDS_HUMAN_APPROVAL	Rollbacks risk data migrations and schema mismatches. A human must approve rollback actions.
Rule 3: Safe Low-Risk	Action is restart (conf ≥ 0.6)	AUTO_APPROVE	Transient memory leaks or worker deadlocks benefit from rapid automated restart without waiting for human wakeup.
Rule 4: Rate Limiting	Action is scale (> 3 per 10m)	REJECTED (Rate Limited)	Prevents runaway auto-scaling storms and cloud bill explosions during cascading traffic anomalies.
Rule 5: Anti-Flapping Gate	Back-to-back action without VERIFY	REJECTED	Prohibits taking consecutive actions without verifying the health outcome of the previous mitigation first.

6. Empirical Evaluation & Benchmark Results

To move beyond marketing hype, SentinelLoop was validated against an empirical suite of **16 distinct fault scenarios** across 4 incident classes. Ground truth was strictly documented before running tests.

Metric Name	Score	Evaluation Context
Diagnosis Accuracy	56.2% (9/16)	Exact match on complex root causes
Correct Gate Rate	75.0% (12/16)	Guardrail chose the correct safety action
False Auto-Action Rate	18.8% (3/16)	Unguarded LLM had 60% false actions
Mean Latency	6.4 seconds	From anomaly detection to gated action
Unit + Integration Tests	52 / 52	48 unit tests + 4 live e2e tests

Key Benchmark Insights

- 1. Where the LLM Succeeded:** For standard memory leaks, CPU thread starvation, and obvious deployment crashes, the LLM achieved near 100% accuracy and proposed instantaneous mitigation.
- 2. Where the LLM Struggled:** In multi-variable scenarios (e.g. an upstream payment gateway timing out while CPU was concurrently high), the LLM suffered from "recency bias", diagnosing CPU overload instead of external dependency timeout.
- 3. The Guardrail Saved the Day:** In 7 out of 7 misdiagnosed cases, the guardrail intercepted the LLM's low-confidence or dangerous action and either rejected it or escalated to a human.

7. Comprehensive Interview Playbook & Talking Points

Q1 "Can you walk me through this project from end to end?"

Recommended Answer Structure:

"SentinelLoop is an autonomous SRE agent I built to solve the trust problem with AI in infrastructure. Modern LLMs are great at analyzing telemetry, but they are too unpredictable to let run wild on production services. I designed a 6-stage finite state machine: Perceive, Diagnose, Plan, Gate, Act, and Verify."

"The agent monitors a microservice, collects logs and metrics, and when latency or error rates spike, feeds the telemetry and matching runbooks to an LLM via NVIDIA NIM. The LLM returns a structured JSON root-cause diagnosis and recommends one action from a closed allow-list. But before that action is touched, it must pass through my Guardrail Engine—a deterministic, rule-based gate written in pure Python. The guardrail enforces policies like confidence thresholds and mandatory human approvals for high-impact actions like rollbacks. Finally, an executor applies approved actions, verifies health, and writes an immutable audit trail."

Q2 "Why did you build a separate Guardrail Engine instead of prompting the LLM to follow safety rules?"

Recommended Answer:

"Relying on an LLM to police itself is an anti-pattern in production engineering. Prompted rules suffer from prompt injection, token length decay, and non-deterministic compliance. An LLM might follow a rule 95% of the time, but in SRE, a 5% failure rate means accidentally dropping production databases or triggering rolling restarts during peak traffic. By encoding safety invariants in deterministic code—with an enforced architectural import boundary preventing the guardrail from touching LLM code—we guarantee safety invariants with mathematical certainty."

Q3 "What were the biggest technical challenges you faced during implementation?"

- **Enforcing Strict JSON Schema Output:** LLMs love to include conversational markdown wrappers. Solved this using Pydantic models with strict field validation, temperature 0, and fallback parsing.
- **State Management & Anti-Flapping:** Preventing the agent from thrashing actions (e.g. scaling up and down repeatedly). Built sliding-window state tracking in the guardrail to rate-limit actions and require verification before follow-up actions.
- **Real-world Benchmark Ground Truth:** Constructing 16 realistic scenarios with pre-committed ground truth to measure true diagnosis and safety gate performance honestly, without cherry-picked data.

7. Interview Playbook (Continued)

Q4

"How did you test the agent?"

Answer: "I built a two-tiered test suite totaling 52 automated tests. The first tier consists of 48 isolated unit tests covering every transition in the state machine, mock telemetry injection, every guardrail policy permutation, rate limiting, and executor safety boundaries. The second tier has 4 live end-to-end integration tests connecting directly to NVIDIA's inference API, exercising real LLM calls against simulated fault scenarios."

Q5

"If you were to deploy this to production at scale (v2), what would you improve?"

- Kubernetes / Prometheus Telemetry Ingestion:** Replace synthetic HTTP polling with Prometheus PromQL queries and OpenTelemetry tracing.
- Multi-Agent Diagnostic Debate:** Use a dual-agent consensus model where one agent proposes the diagnosis and a separate critic agent challenges the hypothesis using historical telemetry.
- Slack / PagerDuty Interactive Webhooks:** Route `NEEDS_HUMAN_APPROVAL` decisions directly into Slack interactive message buttons with 1-click on-call approval.
- RAG Runbook Search:** Implement vector embeddings over thousands of enterprise internal wiki runbooks using hybrid dense/sparse retrieval.

8. Technical Architecture Reference & Code Map

File / Module	Core Responsibility	Key Technical Pattern
<code>agent/orchestrator.py</code>	Main state machine driver	FSM pattern; executes loop across 6 states; handles retries and escalation
<code>agent/guardrail.py</code>	Deterministic policy engine	Zero-LLM dependency rule engine; rate limiting; blast radius containment
<code>agent/diagnosis.py</code>	AI reasoning layer	NVIDIA NIM LLM integration; Pydantic schema parsing; few-shot prompt injection
<code>agent/perception.py</code>	Telemetry ingest & runbook matching	Threshold breach detection; token-efficient runbook context formatting
<code>agent/executor.py</code>	Sandboxed action dispatcher	Closed allow-list execution; zero OS subprocesses; HTTP action client
<code>agent/audit.py</code>	Compliance audit trail	Immutable append-only event logging; timeline reconstruction
<code>toy_service/service.py</code>	Target microservice	FastAPI app simulating latency, memory leaks, error spikes, and bad deploys
<code>benchmark/runner.py</code>	Evaluation harness	Automated 16-scenario batch testing; computes accuracy, gate rates, MTTR

9. Project Metadata & Verification Checklist

GitHub

<https://github.com/GoDxVictoryRR/R-.git>

Branch: `main` (Commit verified & clean)

Python Version: 3.13.7 (Windows / Linux compatible)

Repository: **Human Checkpoints:** All 4 Cleared & Documented

LLM Engine: DeepSeek-V4-Flash via NVIDIA NIM

Evaluation Status: Run v2 Benchmarked & Frozen